



DSA Notes

Order of Growth — Beginner's Cheatsheet



1. What is Order of Growth?

When we write code, we want to know: 'How slow does my code get as the input grows?'
Order of Growth describes the pattern of how fast an algorithm's time (or steps) increases when the input size (n) gets larger.



Think of it this way: If your input doubles, how much more work does your code do?

It focuses on:

- Growth pattern — HOW fast does it grow? ✓
- NOT exact time — we ignore the milliseconds ✗



2. The MOST Important Rule



Keep only the HIGHEST growing term. Throw away everything else.

When you have a complex expression like:

$$f(n) = 2n^2 + n + 6$$

You simplify by:

- Ignoring constants → 2, 6 don't matter
- Ignoring smaller terms → n is tiny compared to n^2
- Keeping the biggest term → n^2 wins!



Final answer: $O(n^2)$



3. Examples to Practice

```
f(n) = 2n2 + n + 6
      ↑ biggest term is n2
      → O(n2)

g(n) = 100n + 3
      ↑ biggest term is n
      → O(n)

h(n) = 5n log n + 20n
      ↑ biggest term is n log n
      → O(n log n)
```

Notice: $100n$ becomes just $O(n)$. The 100 is a constant — we ignore it!

4. Growth Order — Memorize This!

From FASTEST to SLOWEST:

```
O(1) < O(log n) < O(n) < O(n log n) < O(n2) < O(n3) < O(2n)
```

What this means in plain English:

$O(1)$	Runs the same no matter the input size (best!)
$O(\log n)$	Cuts problem in half each step (e.g., binary search)
$O(n)$	One loop through all items (linear)
$O(n \log n)$	Sorting algorithms like merge sort
$O(n^2)$	Nested loops — slows down fast!
$O(2^n)$	Exponential — avoid at all costs! (worst)

5. How to Find Complexity — 3 Steps

Step 1 → Write the Expression

Identify what each piece of code contributes:

- Single loop over n items $\rightarrow n$
- Nested loop (loop inside loop) $\rightarrow n \times n = n^2$
- Each step cuts the problem in half (like doubling i) $\rightarrow \log n$

Step 2 → Combine the Terms

How you combine depends on the structure:

- Separate loops (one after another) $\rightarrow$ ADD them
- Nested loops (one inside another) $\rightarrow$ MULTIPLY them

```
// Separate loops  $\rightarrow O(n) + O(n) = O(n)$ 
for (let i = 0; i < n; i++) { ... }
for (let j = 0; j < n; j++) { ... }

// Nested loops  $\rightarrow O(n) \times O(n) = O(n^2)$ 
for (let i = 0; i < n; i++) {
  for (let j = 0; j < n; j++) { ... }
}
```

Step 3 → Simplify

Pick the highest growing term. Drop constants. Drop smaller terms.



Tip: Always ask — which term grows the fastest? That's your answer.



6. Common Code Patterns

Code Pattern	Complexity
Single loop	$O(n)$
Nested loop	$O(n^2)$
$i *= 2$	$O(\log n)$
$i /= 2$	$O(\log n)$
Nested ($\log \times n$)	$O(n \log n)$

The key insight with $i *= 2$ and $i /= 2$:

```
// i doubles each time: 1, 2, 4, 8, 16 ...
// Takes  $\log_2(n)$  steps to reach n
//  $\rightarrow O(\log n)$ 

for (let i = 1; i < n; i *= 2) {
  console.log(i);
}
```

7. The Addition Rule

When you add two complexities, keep the bigger one:

```
 $O(n + n^2) \rightarrow O(n^2)$  //  $n^2$  dominates
 $O(n + \log n) \rightarrow O(n)$  //  $n$  dominates
 $O(n + n + \log n) \rightarrow O(n)$  // still just  $O(n)$ 
```

 **Why? Because as $n \rightarrow \infty$, the smaller terms become irrelevant.**

8. The Multiplication Rule

When loops are nested, multiply their complexities:

```
Outer loop  $\rightarrow O(n)$ 
Inner loop  $\rightarrow O(n)$ 
-----
Result  $\rightarrow O(n \times n) = O(n^2)$ 
```

```
Outer loop  $\rightarrow O(\log n)$ 
Inner loop  $\rightarrow O(n)$ 
-----
Result  $\rightarrow O(n \log n)$ 
```

9. Important Comparisons

```
log n << n << n log n << n2
```

What '<<' means: 'much much smaller than'. So log n grows WAY slower than n²

Example with n = 1000:

log n	≈ 10 operations
n	1,000 operations
n log n	≈ 10,000 operations
n ²	1,000,000 operations (!)

10. Mental Model — Ask These 3 Questions

? Question 1: How many times does the loop run?

? Question 2: Does the step size change? (linear vs doubling/halving?)

? Question 3: Are the loops nested or separate?

Final Takeaway

Order of Growth = how fast your code slows down when input grows

- Ignore constants ($2n \rightarrow n$)
- Ignore smaller terms ($n^2 + n \rightarrow n^2$)
- Focus on the biggest growth term
- Left side of the chain = faster algorithm = better!